

The routine `expint` allows fast evaluation of $E_n(x)$ to any accuracy EPS within the reach of your machine's precision for floating-point numbers. The only modification required for increased accuracy is to supply Euler's constant with enough significant digits. Wrench [2] can provide you with the first 328 digits if necessary!

`expint.h`

`Doub expint(const Int n, const Doub x)`

Evaluates the exponential integral $E_n(x)$.

```
{
    static const Int MAXIT=100;
    static const Doub EULER=0.577215664901533,
        EPS=numeric_limits<Doub>::epsilon(),
        BIG=numeric_limits<Doub>::max()*EPS;
    Here MAXIT is the maximum allowed number of iterations; EULER is Euler's constant
    γ; EPS is the desired relative error, not smaller than the machine precision; BIG is a
    number near the largest representable floating-point number.

    Int i,ii,nm1=n-1;
    Doub a,b,c,d,del,fact,h,psi,ans;
    if (n < 0 || x < 0.0 || (x==0.0 && (n==0 || n==1)))
        throw("bad arguments in expint");
    if (n == 0) ans=exp(-x)/x;                Special case.
    else {
        if (x == 0.0) ans=1.0/nm1;            Another special case.
        else {
            if (x > 1.0) {                    Lenz's algorithm (§5.2).
                b=x+n;
                c=BIG;
                d=1.0/b;
                h=d;
                for (i=1;i<=MAXIT;i++) {
                    a = -i*(nm1+i);
                    b += 2.0;
                    d=1.0/(a*d+b);            Denominators cannot be zero.
                    c=b+a/c;
                    del=c*d;
                    h *= del;
                    if (abs(del-1.0) <= EPS) {
                        ans=h*exp(-x);
                        return ans;
                    }
                }
                throw("continued fraction failed in expint");
            } else {
                ans = (nm1!=0 ? 1.0/nm1 : -log(x)-EULER);    Evaluate series.
                fact=1.0;                                     Set first term.
                for (i=1;i<=MAXIT;i++) {
                    fact *= -x/i;
                    if (i != nm1) del = -fact/(i-nm1);
                    else {
                        psi = -EULER;                Compute ψ(n).
                        for (ii=1;ii<=nm1;ii++) psi += 1.0/ii;
                        del=fact*(-log(x)+psi);
                    }
                    ans += del;
                    if (abs(del) < abs(ans)*EPS) return ans;
                }
                throw("series failed in expint");
            }
        }
    }
    return ans;
}
```

A good algorithm for evaluating Ei is to use the power series for small x and the asymptotic series for large x . The power series is

$$Ei(x) = \gamma + \ln x + \frac{x}{1 \cdot 1!} + \frac{x^2}{2 \cdot 2!} + \cdots \quad (6.3.10)$$

where γ is Euler's constant. The asymptotic expansion is

$$Ei(x) \sim \frac{e^x}{x} \left(1 + \frac{1!}{x} + \frac{2!}{x^2} + \cdots \right) \quad (6.3.11)$$

The lower limit for the use of the asymptotic expansion is approximately $|\ln \text{EPS}|$, where EPS is the required relative error.

```
Doub ei(const Doub x) { expint.h
Computes the exponential integral Ei(x) for x > 0.
    static const Int MAXIT=100;
    static const Doub EULER=0.577215664901533,
        EPS=numeric_limits<Doub>::epsilon(),
        FPMIN=numeric_limits<Doub>::min()/EPS;
    Here MAXIT is the maximum number of iterations allowed; EULER is Euler's constant  $\gamma$ ; EPS
    is the relative error, or absolute error near the zero of Ei at  $x = 0.3725$ ; FPMIN is a number
    close to the smallest representable floating-point number.
    Int k;
    Doub fact,prev,sum,term;
    if (x <= 0.0) throw("Bad argument in ei");
    if (x < FPMIN) return log(x)+EULER;
    if (x <= -log(EPS)) {
        sum=0.0;
        fact=1.0;
        for (k=1;k<=MAXIT;k++) {
            fact *= x/k;
            term=fact/k;
            sum += term;
            if (term < EPS*sum) break;
        }
        if (k > MAXIT) throw("Series failed in ei");
        return sum+log(x)+EULER;
    } else {
        sum=0.0;
        term=1.0;
        for (k=1;k<=MAXIT;k++) {
            prev=term;
            term *= k/x;
            if (term < EPS) break;
            Since final sum is greater than one, term itself approximates the relative error.
            if (term < prev) sum += term;
            else {
                sum -= prev;
                break;
            }
        }
        return exp(x)*(1.0+sum)/x;
    }
}
```

Special case: Avoid failure of convergence test because of underflow.
Use power series.

Use asymptotic series.
Start with second term.

Still converging: Add new term.

Diverging: Subtract previous term and exit.

CITED REFERENCES AND FURTHER READING:

Stegun, I.A., and Zucker, R. 1974, "Automatic Computing Methods for Special Functions. II. The Exponential Integral $E_n(x)$," *Journal of Research of the National Bureau of Standards*,

- vol. 78B, pp. 199–216; 1976, “Automatic Computing Methods for Special Functions. III. The Sine, Cosine, Exponential Integrals, and Related Functions,” *op. cit.*, vol. 80B, pp. 291–311.
- Amos D.E. 1980, “Computation of Exponential Integrals,” *ACM Transactions on Mathematical Software*, vol. 6, pp. 365–377[1]; also vol. 6, pp. 420–428.
- Abramowitz, M., and Stegun, I.A. 1964, *Handbook of Mathematical Functions* (Washington: National Bureau of Standards); reprinted 1968 (New York: Dover); online at <http://www.nist.gov/aands>, Chapter 5.
- Wrench J.W. 1952, “A New Calculation of Euler’s Constant,” *Mathematical Tables and Other Aids to Computation*, vol. 6, p. 255.[2]

6.4 Incomplete Beta Function

The incomplete beta function is defined by

$$I_x(a, b) \equiv \frac{B_x(a, b)}{B(a, b)} \equiv \frac{1}{B(a, b)} \int_0^x t^{a-1} (1-t)^{b-1} dt \quad (a, b > 0) \quad (6.4.1)$$

It has the limiting values

$$I_0(a, b) = 0 \quad I_1(a, b) = 1 \quad (6.4.2)$$

and the symmetry relation

$$I_x(a, b) = 1 - I_{1-x}(b, a) \quad (6.4.3)$$

If a and b are both rather greater than one, then $I_x(a, b)$ rises from “near-zero” to “near-unity” quite sharply at about $x = a/(a+b)$. Figure 6.4.1 plots the function for several pairs (a, b) .

The incomplete beta function has a series expansion

$$I_x(a, b) = \frac{x^a (1-x)^b}{a B(a, b)} \left[1 + \sum_{n=0}^{\infty} \frac{B(a+1, n+1)}{B(a+b, n+1)} x^{n+1} \right] \quad (6.4.4)$$

but this does not prove to be very useful in its numerical evaluation. (Note, however, that the beta functions in the coefficients can be evaluated for each value of n with just the previous value and a few multiplies, using equations 6.1.9 and 6.1.3.)

The continued fraction representation proves to be much more useful:

$$I_x(a, b) = \frac{x^a (1-x)^b}{a B(a, b)} \left[\frac{1}{1 + \frac{d_1}{1 + \frac{d_2}{\dots}}} \right] \quad (6.4.5)$$

where

$$\begin{aligned} d_{2m+1} &= -\frac{(a+m)(a+b+m)x}{(a+2m)(a+2m+1)} \\ d_{2m} &= \frac{m(b-m)x}{(a+2m-1)(a+2m)} \end{aligned} \quad (6.4.6)$$

This continued fraction converges rapidly for $x < (a+1)/(a+b+2)$, except when a and b are both large, when it can take $O(\sqrt{\min(a, b)})$ iterations. For $x >$

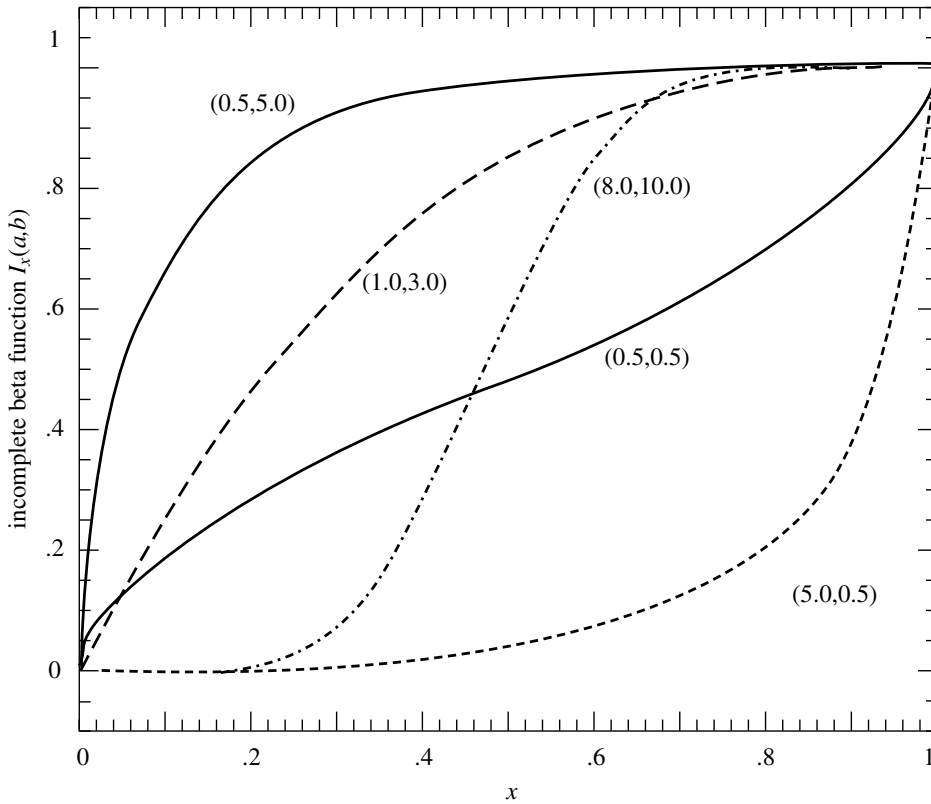


Figure 6.4.1. The incomplete beta function $I_x(a, b)$ for five different pairs of (a, b) . Notice that the pairs $(0.5, 5.0)$ and $(5.0, 0.5)$ are symmetrically related as indicated in equation (6.4.3).

$(a + 1)/(a + b + 2)$ we can just use the symmetry relation (6.4.3) to obtain an equivalent computation in which the convergence is again rapid. Our computational strategy is thus very similar to that used in Gamma: We use the continued fraction except when a and b are both large, in which case we do a single step of high-order Gauss-Legendre quadrature.

Also as in Gamma, we code an inverse function using Halley's method. When a and b are both ≥ 1 , the initial guess comes from §26.5.22 in reference [1]. When either is less than 1, the guess comes from first crudely approximating

$$\int_0^1 t^{a-1}(1-t)^{b-1} dt \approx \frac{1}{a} \left(\frac{a}{a+b} \right)^a + \frac{1}{b} \left(\frac{b}{a+b} \right)^b \equiv S \quad (6.4.7)$$

which comes from breaking the integral at $t = a/(a+b)$ and ignoring one factor in the integrand on each side of the break. We then write

$$I_x(a, b) \approx \begin{cases} x^a/(Sa) & x \leq a/(a+b) \\ (1-x)^b/(Sb) & x > a/(a+b) \end{cases} \quad (6.4.8)$$

and solve for x in the respective regimes. While crude, this is good enough to get well within the basin of convergence in all cases.

ncgammabeta.h

struct Beta : Gauleg18 {

Object for incomplete beta function. Gauleg18 provides coefficients for Gauss-Legendre quadrature.

```
static const Int SWITCH=3000;           When to switch to quadrature method.
static const Doub EPS, FPMIN;           See end of struct for initializations.
```

```
Doub betai(const Doub a, const Doub b, const Doub x) {
```

Returns incomplete beta function $I_x(a, b)$ for positive a and b , and x between 0 and 1.

```
    Doub bt;
    if (a <= 0.0 || b <= 0.0) throw("Bad a or b in routine betai");
    if (x < 0.0 || x > 1.0) throw("Bad x in routine betai");
    if (x == 0.0 || x == 1.0) return x;
    if (a > SWITCH && b > SWITCH) return betaiapprox(a,b,x);
    bt=exp(gammln(a+b)-gammln(a)-gammln(b)+a*log(x)+b*log(1.0-x));
    if (x < (a+1.0)/(a+b+2.0)) return bt*betacf(a,b,x)/a;
    else return 1.0-bt*betacf(b,a,1.0-x)/b;
}
```

```
Doub betacf(const Doub a, const Doub b, const Doub x) {
```

Evaluates continued fraction for incomplete beta function by modified Lentz's method (§5.2). User should not call directly.

```
    Int m,m2;
    Doub aa,c,d,del,h,qab,qam,qap;
    qab=a+b;
    qap=a+1.0;
    qam=a-1.0;
    c=1.0;
    d=1.0-qab*x/qap;
    if (fabs(d) < FPMIN) d=FPMIN;
    d=1.0/d;
    h=d;
    for (m=1;m<10000;m++) {
        m2=2*m;
        aa=m*(b-m)*x/((qam+m2)*(a+m2));
        d=1.0+aa*d;
        if (fabs(d) < FPMIN) d=FPMIN;
        c=1.0+aa/c;
        if (fabs(c) < FPMIN) c=FPMIN;
        d=1.0/d;
        h *= d*c;
        aa = -(a+m)*(qab+m)*x/((a+m2)*(qap+m2));
        d=1.0+aa*d;
        if (fabs(d) < FPMIN) d=FPMIN;
        c=1.0+aa/c;
        if (fabs(c) < FPMIN) c=FPMIN;
        d=1.0/d;
        del=d*c;
        h *= del;
        if (fabs(del-1.0) <= EPS) break;
    }
    return h;
}
```

These q's will be used in factors that occur in the coefficients (6.4.6).

First step of Lentz's method.

One step (the even one) of the recurrence.

Next step of the recurrence (the odd one).

Are we done?

```
Doub betaiapprox(Doub a, Doub b, Doub x) {
```

Incomplete beta by quadrature. Returns $I_x(a, b)$. User should not call directly.

```
    Int j;
    Doub xu,t,sum,ans;
    Doub a1 = a-1.0, b1 = b-1.0, mu = a/(a+b);
    Doub lnmu=log(mu),lnmuc=log(1.-mu);
    t = sqrt(a*b/(SQR(a+b)*(a+b+1.0)));
    if (x > a/(a+b)) {
        if (x >= 1.0) return 1.0;
        xu = MIN(1.,MAX(mu + 10.*t, x + 5.0*t));
    } else {
```

Set how far to integrate into the tail: